EXERCISE-6

Q1)Define an interface User with the following properties:
id(number), name(string), email(string), age(number optional)

```ts
interface User{
    id: number,
    name: string,
    email: string,
    age?: number;
}

const user1: User = {
    id:101,
    name:'Harsh',
    email:'xyz@gmail.com',
    age:21,
}

console.log(user1);
```

Output:

```
harsh-raj@TTNPL-harsh:js_exercise6 (js-assignment)$ node dist/q1.js
{ id: 101, name: 'Harsh', email: 'xyz@gmail.com', age: 21 }
```

Q2)Create a class UserManager with:

- **A private array users: User[] to store user data.**
- **A method addUser(user: User): void that adds a new user.**
- **A method removeUser(id: number): void that removes a user by ID.**
- **A method getUser(id: number): User | undefined that retrieves a user by ID.**
- **A method getAllUsers(): User[] that returns all users.**

TS q2.ts > UserManager

```ts
interface User{
    id: number,
    name: string,
    address: string,
    age?: number;
}

class UserManager{
    private users: User[]= [];

    addUser(user: User): void{
        this.users.push(user);
    }

    removeUser(id: number): void{
        this.users.forEach(user=>{
            if(user.id == id){
                let x=this.users.indexOf(user);
                this.users.splice(x,1);
            }
        })
    }

    getUserbyId(id: number): User|undefined{
        for(const user of this.users){
            if(user.id===id) return user
        }
        return;
    }

    getAllUsers() : User[]{
        return this.users;
    }

}
```

```
const admin = new UserManager();

admin.addUser({id:101,name: 'Harsh',address: 'Delhi'});
admin.addUser({id:102,name: 'Chitra',address: 'Ghaziabad',age:30});
admin.addUser({id:103,name: 'Vaibhav',address: 'Noida',age: 21});
admin.addUser({id:104,name: 'Manas',address: 'Gurugram'});

console.log(admin.getUserbyId(101));

admin.removeUser(101);

console.log(admin.getAllUsers());
```

Output:

```
harsh-raj@TTNPL-harsh:js_exercise6 (js-assignment)$ node dist/q2.js
{ id: 101, name: 'Harsh', address: 'Delhi' }
[
  { id: 102, name: 'Chitra', address: 'Ghaziabad', age: 30 },
  { id: 103, name: 'Vaibhav', address: 'Noida', age: 21 },
  { id: 104, name: 'Manas', address: 'Gurugram' }
]
```

Q3)Use Arrow Functions & Default Parameters.Add a method getUser = (name: string = "Guest"): string that returns a greeting message.

```
4
5       //Question 3
6       getUser=(name: string="Guest"):string=>{
7           return `Good Morning ${name}`;
8       }
9   }
0
```

```
54  //Question 3
55  console.log(admin.getUser());
```

Output:

```
harsh-raj@TTNPL-harsh:js_exercise6 (js-assignment)$ node dist/q2.js
Good Morning Guest
```

Q4)Use Destructuring & Spread Operator
Create a function printUserDetails(user: User): void that logs user details using object destructuring.

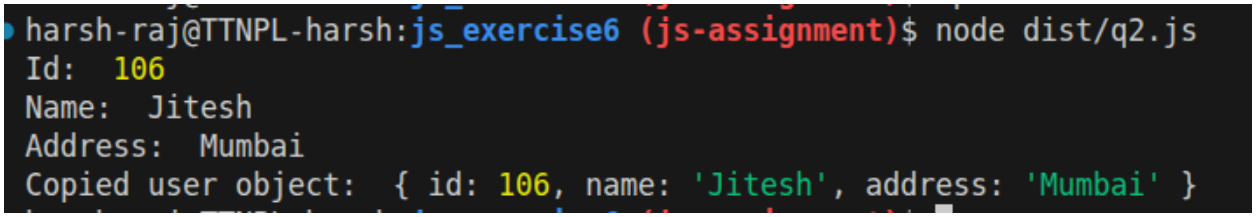
```
//Question 4
printUserDetails(user:User):void{
    const Usercopy={...user};
    const {id,name,address}=user;

    console.log("Id: ", id)
    console.log("Name: ", name);
    console.log("Address: ", address);

    console.log("Copied user object: ", Usercopy);
}
```

```
//Question 4
const user1: User={
    id:106,
    name:'Jitesh',
    address:'Mumbai'
}
const userID=admin.getUserbyId(102);
admin.printUserDetails(user1);
```

Output:


harsh-raj@TTNPL-harsh:js_exercise6 (js-assignment)$ node dist/q2.js
Id: 106
Name: Jitesh
Address: Mumbai
Copied user object: { id: 106, name: 'Jitesh', address: 'Mumbai' }